Deployment-Oriented Evaluation of Self-Hosted, Containerized, and Managed LLM Serving

Taixing Bi
Independent Researcher

***Abstract*—*Deploying large language models (LLMs) in production requires balancing latency, throughput, serving cost, and operational complexity. Existing research focuses mainly on model optimization and inference acceleration, leaving little empirical guidance for choosing among deployment strategies. This paper presents a deployment-oriented evaluation of three representative LLM serving approaches: self-hosted vLLM on local GPUs, containerized deployment on Amazon ECS, and managed inference via Amazon Bedrock. Using the same Qwen2.5-7B model and identical benchmark workloads, we measure end-to-end latency, time-to-first-token (TTFT), throughput, and normalized serving cost across varying concurrency, prompt length, and output length. Increasing concurrency substantially improves serving efficiency for all three strategies, lowering normalized cost per request while raising throughput. Self-hosted deployment achieves the lowest serving cost and competitive latency under moderate load, making it well suited to stable production services with high utilization. Amazon Bedrock delivers the highest throughput and the most stable TTFT as concurrency grows, but at the highest normalized cost. Amazon ECS falls between the two, trading deployment flexibility for intermediate cost efficiency. Rather than naming a universally optimal strategy, this work shows that deployment choice should follow workload characteristics and operational requirements, and offers practical guidance for selecting LLM deployment architectures in production.*

***Index Terms*—*Large Language Models, LLM Serving, Deployment Evaluation, vLLM, Amazon Bedrock, Amazon ECS, Throughput, Latency, Cost Analysis*

I. INTRODUCTION

Large language models now underpin a wide range of AI applications — conversational assistants, retrieval-augmented generation (RAG), software development tools, customer support, and autonomous agents. As model capability improves, serving these systems efficiently in production has become an increasingly important engineering problem.

Recent work has substantially improved LLM serving efficiency through continuous batching, memory paging, KV-cache optimization, and distributed inference. Frameworks such as vLLM, TensorRT-LLM, and SGLang have cut inference latency and raised GPU utilization considerably. But this research largely asks how to optimize inference on a given infrastructure — not which deployment strategy to choose in the first place.

Organizations deploying LLMs today face a concrete but poorly answered question: given a fixed model, should inference run on self-hosted GPUs, a cloud-managed container, or a fully managed inference service? Vendor blog posts and marketing benchmarks address pieces of this question, but rarely under identical models, workloads, and cost accounting — making cross-platform comparison unreliable in practice.

Organizations today typically deploy LLMs one of three ways: self-hosted deployment, running inference servers on dedicated, organization-managed GPUs; containerized cloud deployment, such as Amazon ECS, which keeps infrastructure control with the user while adding cloud elasticity; and managed inference services, such as Amazon Bedrock, which abstract infrastructure management away entirely. Each option trades off latency, throughput, cost, scalability, and operational complexity differently.

This paper offers a deployment-oriented evaluation of representative serving architectures. Using the same Qwen2.5-7B model [15], identical benchmark workloads, and a unified methodology, we compare self-hosted vLLM, Amazon ECS, and Amazon Bedrock across concurrency, prompt length, and output length — measuring end-to-end latency, TTFT, throughput, and normalized cost so the environments can be compared consistently despite their very different infrastructure.

This paper makes four contributions:

1) A unified evaluation protocol for comparing self-hosted, containerized, and managed LLM serving under consistent models, workloads, and streaming behavior.

2) A unified cost-accounting framework that reconciles three incompatible billing models — electricity-based self-hosting, hourly cloud instances, and Custom-Model-Unit billing with minimum windows — through normalized and session-allocated cost metrics.

3) A controlled empirical study across concurrency, prompt length, and output length, covering throughput, TTFT, end-to-end latency, and cost.

4) Scenario-based deployment guidance for practitioners choosing among these strategies under different production conditions.

II. RELATED WORK

A. LLM Serving Systems

Efficient LLM serving is hard because autoregressive generation carries high memory requirements, variable sequence lengths, and an inherent tension between latency and throughput. Orca introduced iteration-level scheduling [1]. vLLM tackled KV-cache fragmentation via PagedAttention

combined with continuous batching [2]. Sarathi and Sarathi-Serve addressed prefill/decode interaction with chunked prefill and stall-free scheduling [3], [4]. SGLang added RadixAttention for prefix cache reuse [5]. TensorRT-LLM contributes optimized kernels, quantization, and speculative decoding [6]. More recently, disaggregated serving systems assign prefill and decode to separate GPU pools to eliminate phase interference and scale each phase independently [16]. Together these systems establish memory management, batching, scheduling, and kernel optimization as key levers for efficient inference — but they evaluate improvements within a single serving engine or fixed hardware environment. We instead ask how the same model behaves across different deployment and ownership models.

B. Resource Management and Workload-Aware Serving

AlpaServe showed that model parallelism supports statistical multiplexing across models for bursty traffic [7], highlighting a trade-off between parallelism overhead and pooling gains. This matters for deployment selection because traffic intensity and burstiness affect both utilization and cost. Our study complements this line of work by treating concurrency, prompt length, and output length as workload variables and measuring how each affects TTFT, latency, throughput, and cost across three deployment strategies.

C. LLM Inference Benchmarking

MLPerf Inference established principles for architecture-neutral, reproducible comparison of ML inference systems [8]. LLMPerf and NVIDIA GenAI-Perf provide finer-grained serving metrics such as TTFT and token throughput through a common client interface [9], [10]; AWS has used LLMPerf to benchmark Bedrock without exposing serving configuration [11]. We build on these methodologies but compare three operationally distinct deployment modes of the same model rather than ranking accelerators or engines.

D. Cloud and Managed LLM Deployment

Amazon ECS supports GPU-enabled containers on EC2 G-series instances, leaving serving-framework and hardware decisions to the user [12]. Amazon Bedrock Custom Model Import is more managed, hiding the accelerator and serving infrastructure while billing by model copies, Custom Model Units, and five-minute windows rather than per-token pricing [13], [14]. Industry cost analyses report that self-hosted infrastructure can offer substantially lower cost per token than managed API access under sustained utilization, though the break-even point depends heavily on utilization rate and hardware lifecycle [17]. Official documentation covers each platform's capabilities and billing individually, but none offers a controlled comparison using identical weights, request distributions, streaming behavior, and cost normalization.

E. Positioning of This Work

This study addresses a practical question neither serving-system research nor existing benchmarking frameworks answer: given the same model and workload, when should a system use dedicated self-hosted hardware, a cloud-hosted GPU container, or a managed inference service? We distinguish this work through four choices: the same Qwen2.5-7B model across all deployment modes; workloads controlled by concurrency, prompt length, and output length; streaming metrics covering TTFT, latency, and throughput; and costs normalized across electricity-based, hourly-instance, and Custom Model Unit billing models.

III. EXPERIMENTAL SETUP

A. Evaluation Objective

We evaluate three deployment strategies for serving the same model: self-hosted deployment using vLLM on a dedicated NVIDIA RTX 3090; cloud-containerized deployment using vLLM on an Amazon ECS GPU instance; and managed deployment using Amazon Bedrock Custom Model Import. All three serve the same Qwen2.5-7B-Instruct model through streaming endpoints; a shared benchmark harness submits equivalent requests and records TTFT, end-to-end latency, output-token throughput, completion status, and cost.

B. Deployment Environments

TABLE I. DEPLOYMENT CONFIGURATIONS

Deployment	Serving Framework	Hardware / Service	Pricing Model
Self-host	vLLM	NVIDIA RTX 3090 (24 GB)	Electricity only (350 W, $0.15/kWh)
Amazon ECS	vLLM on ECS	g5.xlarge (NVIDIA A10G)	$1.006/hour
Amazon Bedrock	Custom Model Import	Managed (hardware abstracted)	2 CMUs ($6.8616/hour)

Self-hosted and ECS both use vLLM for OpenAI-compatible streaming inference; Bedrock uses Custom Model Import plus a streaming adapter so TTFT can be collected consistently across all three.

C. Model and Request Configuration

TABLE II. COMMON INFERENCE SETTINGS

Parameter	Value
Model	Qwen2.5-7B-Instruct
Framework	OpenAI-compatible API
Streaming	Enabled (SSE)
Temperature	0
Max output tokens	1024
Prompt length sweep	512–4096 tokens

Parameter	Value
Output length sweep	128–1024 tokens
Concurrency sweep	1, 2, 4, 8, 16, 32, 64
Benchmark tool	LLMPerf
Metrics	TTFT, E2E latency, Throughput, Cost

All experiments use Qwen2.5-7B-Instruct, with the same request format, prompt-generation procedure, output-token limit, and streaming behavior across deployments. We record the actual input and output token counts returned by each endpoint rather than relying only on configured limits.

D. Workload Design

Concurrency sweep: $c \in \{1, 2, 4, 8, 16, 32, 64\}$, prompt length and output length fixed (Figures 1–5). Prompt-length sweep: ~512 to ~4,096 input tokens, concurrency and output length fixed (Figures A1–A4). Output-length sweep: $o \in \{128, 256, 512, 1{,}024\}$ tokens, prompt length and concurrency fixed (Figures A5–A7).

E. Performance Metrics

Time to first token: $TTFT_i = t_{\text{first-token}}(i) - t_{\text{request-start}}(i)$, reported as median (P50). End-to-end latency: $L_i = t_{\text{final-token}}(i) - t_{\text{request-start}}(i)$, reported as P50. Output-token throughput: sum of output tokens over successful requests divided by elapsed time. Request success rate: $N_{\text{successful}} / N_{\text{submitted}}$.

F. Cost Model

Self-hosting: $C = (P_{\text{watts}}/1000) \times C_{\text{kWh}}$; at 350 W and \$0.15/kWh, \$0.0525/hour (electricity only). ECS: \$1.006/hour. Bedrock: $C = N_{\text{copies}} \times N_{\text{CMU}} \times C_{\text{CMU/min}} \times 60$; at one copy, two CMUs, \$0.05718/CMU-min, \$6.8616/hour. We report normalized cost (active-duration cost per successful request) and session-allocated cost (full provisioned-duration cost per successful request) separately, since they answer different questions about compute efficiency versus realized deployment cost.

IV. RESULTS

We organize the results around three research questions: (RQ1) how does concurrency affect throughput and interactive responsiveness; (RQ2) how do the deployment strategies compare on latency–cost and throughput–cost efficiency; (RQ3) how sensitive are the systems to prompt length and output length. All values come from successful streaming requests; since the figures show point estimates without confidence intervals (single-run sweeps, see Section VI), we describe observed differences as directional rather than statistically conclusive, except where effect sizes are large (e.g., >10× throughput changes from concurrency scaling).

A. RQ1: Concurrency Effects

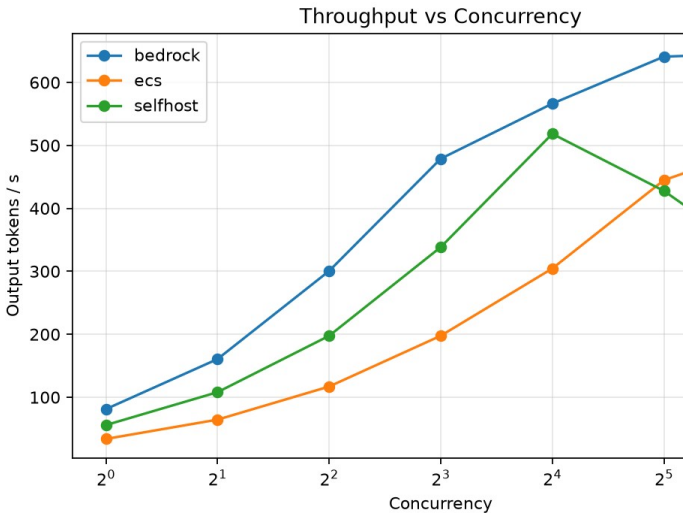

Fig. 1. Output-token throughput vs. concurrency for self-hosted, ECS, and Bedrock deployments.

At concurrency 1, throughput is ~80 tokens/s for Bedrock, ~55 for self-host, and ~35 for ECS. Bedrock scales from ~80 to ~640–650 tokens/s (32→64), plateauing after 32. ECS climbs from ~35 to ~505 tokens/s with no clear plateau through 64. Self-host scales efficiently to concurrency 16 (~55→~520 tokens/s), then declines to ~430 (c=32) and ~305 (c=64), indicating single-GPU saturation.

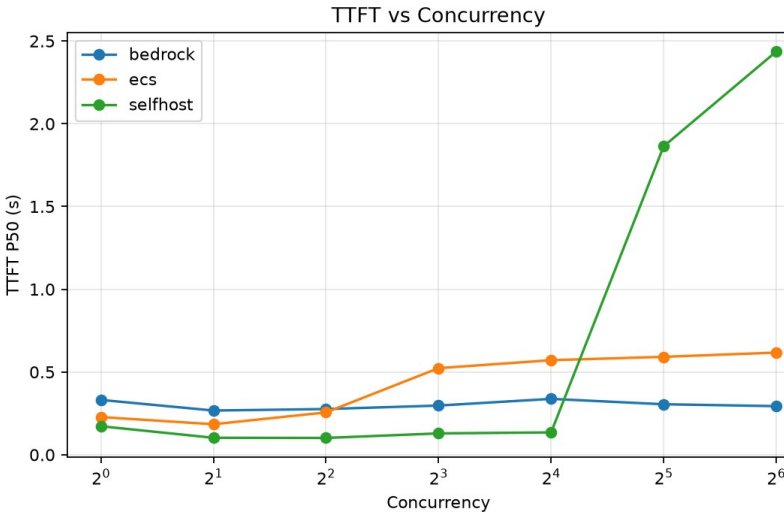

Fig. 2. Median time-to-first-token vs. concurrency.

Bedrock TTFT stays stable at ~0.27–0.35 s across the range. ECS grows gradually from ~0.22 s to ~0.62 s. Self-host gives the lowest TTFT at moderate concurrency (~0.10–0.14 s, c=2–16), then jumps to ~1.87 s (c=32) and ~2.44 s (c=64), coinciding with its throughput decline.

B. RQ2: Cost–Performance Trade-offs

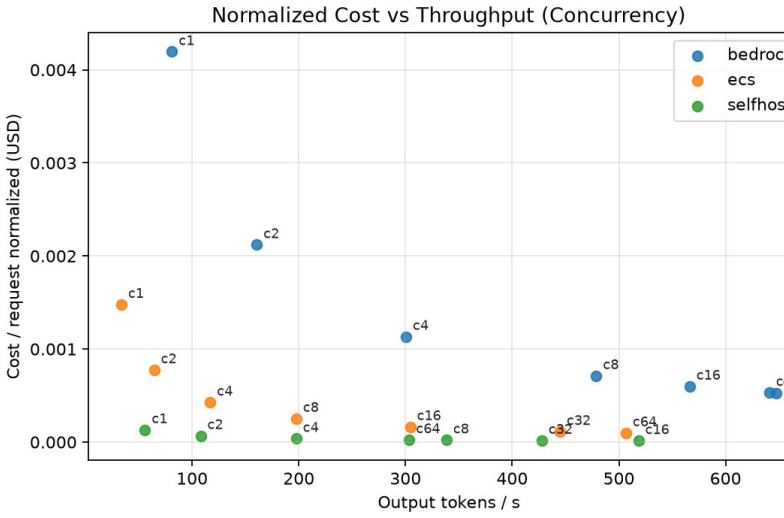

Fig. 3. Normalized cost per successful request vs. output-token throughput.

Bedrock's per-request cost falls from $0.0042 (c=1) to $0.0005–$0.0006 (c=16–64), a ~7.6× improvement as its high hourly cost is spread over more requests. ECS falls from $0.0015 to under $0.0001. Self-host is cheapest throughout ($0.0001 down to $0.00001–$0.00004), reflecting its low assumed electricity-only hourly rate.

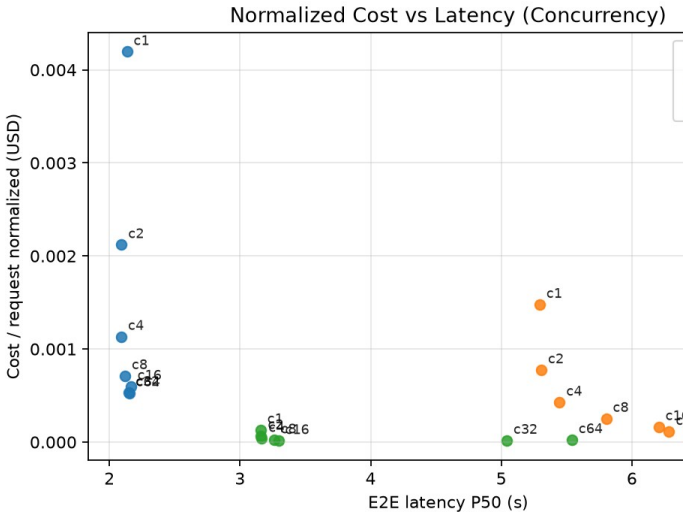

Fig. 4. Normalized cost per successful request vs. median end-to-end latency.

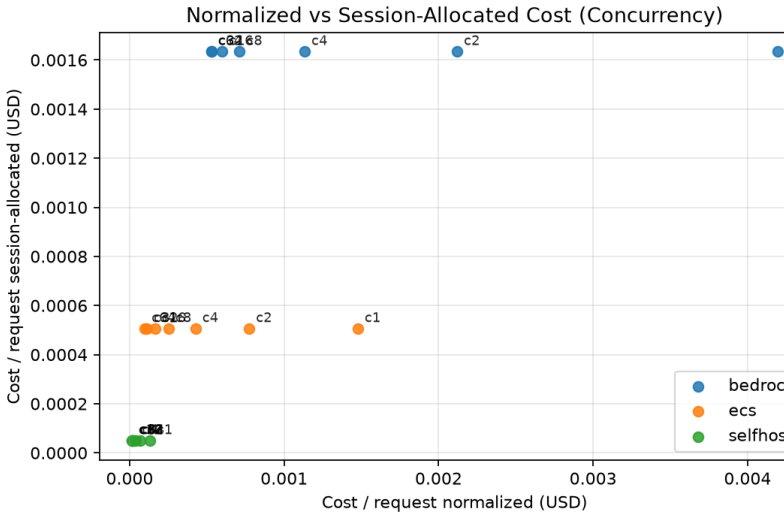

Fig. 5. Normalized vs. session-allocated cost per request across the concurrency sweep.

Bedrock occupies the low-latency, higher-cost region (E2E ~2.1–2.2 s) while cost falls with concurrency. ECS trades latency for cost (5.3 s → 7.1 s as cost falls). Self-host holds ~3.1–3.3 s through c=16 then rises to ~5.5 s at c=64; the c=32/64 points are Pareto-dominated by c=16.

Session-allocated cost is nearly flat across concurrency for each deployment (~$0.00163 Bedrock, ~$0.00050 ECS, ~$0.000045 self-host), while normalized cost varies substantially with concurrency — the two metrics answer different questions about compute efficiency versus realized deployment cost.

C. RQ3: Prompt- and Output-Length Sensitivity

Prompt length has a much smaller effect than concurrency: across ~512–4,096 input tokens, E2E latency moves only ~4.8% (Bedrock) to ~9% (ECS) (Fig. A1). Deployment choice separates the three strategies far more than prompt length does within this range (Figs. A2–A3). Output length has a stronger effect: longer generations raise both latency and cost roughly linearly with decode steps across all three deployments (Figs. A5–A6), while throughput within a deployment stays comparatively stable across output lengths (Fig. A6).

D. Summary of Main Results

TABLE III. DEPLOYMENT DECISION MATRIX

Scenario	Recommended Deployment	Reason
MVP	Bedrock	No infrastructure management, fastest deployment
Stable production	Self-host	Lowest cost under sustained utilization
Enterprise cloud	ECS	Operational control with cloud scalability
Bursty traffic	Bedrock	Stable latency without capacity planning
Cost-sensitive service	Self-host	Lowest normalized serving cost
Compliance / custom runtime	ECS	Full infrastructure and networking control

TABLE IV. QUANTITATIVE COMPARISON ACROSS DEPLOYMENTS

Metric	Self-host	ECS	Bedrock
Peak throughput (tokens/s)	~520	~505	~650
Lowest TTFT (s)	~0.10	~0.18	~0.27
TTFT at c=64 (s)	~2.44	~0.62	~0.29
Lowest normalized cost ($/request)	~0.00001	~0.00009	~0.00050
Best concurrency	16	64	64

Four conclusions follow. First, concurrency is the dominant workload factor, driving most throughput gains and cost reductions while also exposing saturation. Second, self-hosting gives the lowest electricity-based cost and lowest TTFT under moderate concurrency, but that advantage reverses once throughput falls and TTFT spikes at high concurrency. Third,

Bedrock gives the highest throughput and steadiest latency at the highest normalized and session-allocated cost. Fourth, ECS sits in the middle on cost and scales gradually without Bedrock's latency or self-host's cost efficiency, but avoids the abrupt saturation seen on the single RTX 3090.

V. CONCLUSION

This paper evaluated three representative strategies for serving the same LLM under a unified methodology and cost-accounting framework. No single strategy wins across all objectives: Bedrock reaches the highest measured throughput (~650 output tokens/s) with stable TTFT near 0.3 s, at the highest normalized and session-allocated cost. Self-hosting achieves the lowest normalized cost and lowest TTFT under moderate concurrency but degrades beyond c=16. ECS holds an intermediate cost position with gradual scaling and no abrupt saturation in the tested range.

These results support a scenario-based decision framework: self-hosting suits stable, continuously utilized workloads within a known capacity envelope; Bedrock suits applications prioritizing throughput and stable responsiveness over minimum cost; ECS suits cloud-native systems needing runtime control and future horizontal scaling. A key methodological takeaway is that normalized compute cost and realized session cost should be reported separately, since either alone can misrepresent the real economics of continuously provisioned LLM infrastructure.

VI. LIMITATIONS

This study evaluates one 7B model, one self-hosted GPU, one ECS GPU instance, one cloud region, and synthetic workloads. Self-hosted and ECS deployments use different accelerators (RTX 3090 vs. A10G) rather than identical hardware, since our goal is to compare deployment and ownership models as practitioners actually encounter them — a self-hosted GPU purchase and a cloud GPU instance are rarely the same silicon — not to isolate hardware effects in controlled fashion. This means observed differences conflate deployment model with hardware; we do not attempt to separate these factors, and future work using matched hardware across self-hosted and ECS deployments could isolate the deployment-model effect independent of accelerator choice. Bedrock's hidden accelerator and scheduler further prevent hardware-level attribution there. The self-hosted cost model covers electricity only, not hardware amortization, maintenance, cooling, redundancy, or engineering labor.

Given the cost and time of running full concurrency/prompt/output sweeps repeatedly per deployment on both cloud and dedicated hardware, we report single-run point estimates rather than repeated-trial statistics. Where differences are small (e.g., the 4.8%–9% latency change across the prompt-length sweep), we treat them as directional rather than conclusive, and larger effects (e.g., the >10× throughput gains from concurrency scaling) as robust to this limitation. Autoscaling, multi-replica serving, cold starts, failure recovery, and real production traffic were not evaluated.

Future work should extend this benchmark to multiple model sizes and architectures, multi-GPU and multi-replica deployments, autoscaling policies, longer-context workloads, bursty production traces, matched hardware for the self-host/ECS comparison, and cost per successful request under explicit latency and reliability SLOs. A particularly useful extension would evaluate hybrid deployment, where stable baseline traffic runs on self-hosted infrastructure and overflow traffic routes to cloud or managed capacity.

REFERENCES

[1] G.-I. Yu et al., "Orca: A distributed serving system for Transformer-based generative models," in Proc. 16th USENIX OSDI, 2022, pp. 521–538.

[2] W. Kwon et al., "Efficient memory management for large language model serving with PagedAttention," in Proc. 29th SOSP, 2023, pp. 611–626.

[3] A. Agrawal et al., "SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills," arXiv:2308.16369, 2023.

[4] A. Agrawal et al., "Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve," in Proc. 18th USENIX OSDI, 2024, pp. 117–134.

[5] L. Zheng et al., "SGLang: Efficient execution of structured language model programs," in Adv. Neural Inf. Process. Syst., vol. 37, 2024.

[6] NVIDIA, "TensorRT-LLM: A library for optimizing large language model inference," 2023. [Online]. Available: https://github.com/NVIDIA/TensorRT-LLM

[7] Z. Li et al., "AlpaServe: Statistical multiplexing with model parallelism for deep learning serving," in Proc. 17th USENIX OSDI, 2023, pp. 663–679.

[8] V. J. Reddi et al., "MLPerf inference benchmark," in Proc. ACM/IEEE 47th ISCA, 2020.

[9] Ray Project, "LLMPerf: A tool for evaluating the performance of LLM APIs." [Online]. Available: https://github.com/ray-project/llmperf

[10] NVIDIA, "GenAI-Perf: Generative AI performance benchmarking tool." [Online]. Available: https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/perf_analyzer/genai-perf/README.html

[11] Amazon Web Services, "Benchmarking customized models on Amazon Bedrock using LLMPerf and LiteLLM," 2025. [Online]. Available: https://aws.amazon.com/blogs/machine-learning/benchmarking-customized-models-on-amazon-bedrock-using-llmperf-and-litellm/

[12] Amazon Web Services, "Amazon ECS task definitions for GPU workloads." [Online]. Available: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/ecs-gpu.html

[13] Amazon Web Services, "Amazon Bedrock Custom Model Import." [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/userguide/model-customization-import-model.html

[14] Amazon Web Services, "Calculate the cost of running an imported custom model." [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/userguide/import-model-calculate-cost.html

[15] Qwen Team, "Qwen2.5 Technical Report," arXiv:2412.15115, 2024.

[16] Y. Zhong et al., "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in Proc. 18th USENIX OSDI, 2024, pp. 193–210.

[17] Lenovo, "On-Premise vs Cloud: Generative AI Total Cost of Ownership (2025 Edition)," Lenovo Press, 2025. [Online]. Available: https://lenovopress.lenovo.com/lp2225.pdf

APPENDIX

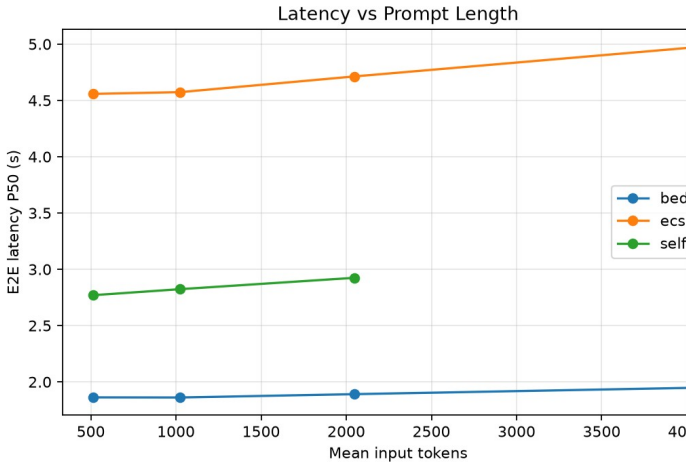

Fig. A1. Median end-to-end latency vs. mean input token count.

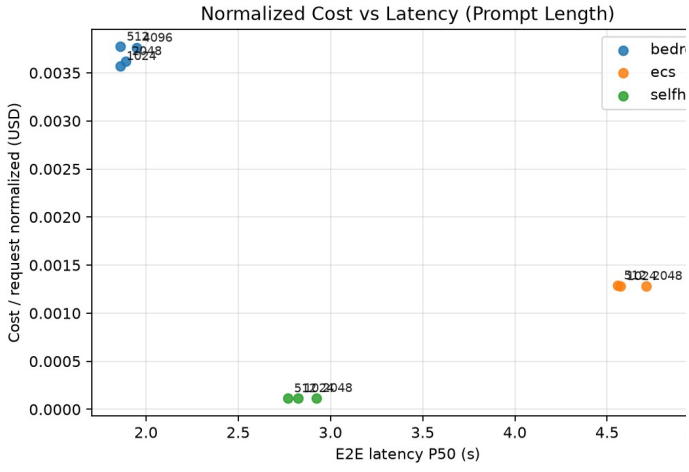

Fig. A2. Normalized cost per request vs. median end-to-end latency, prompt-length sweep.

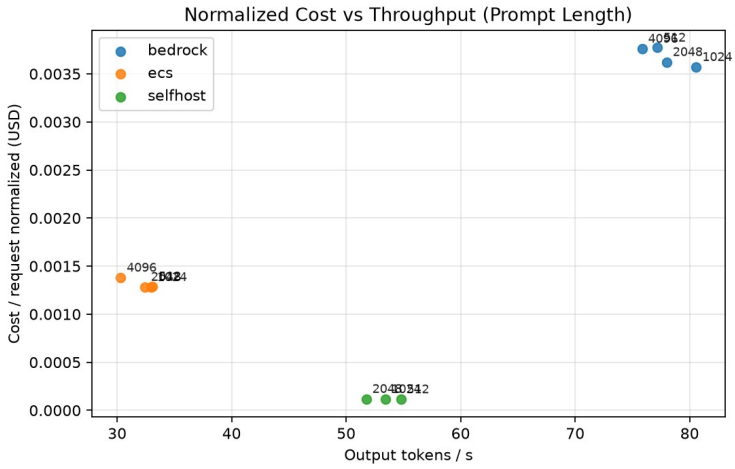

Fig. A3. Normalized cost per request vs. output-token throughput, prompt-length sweep.

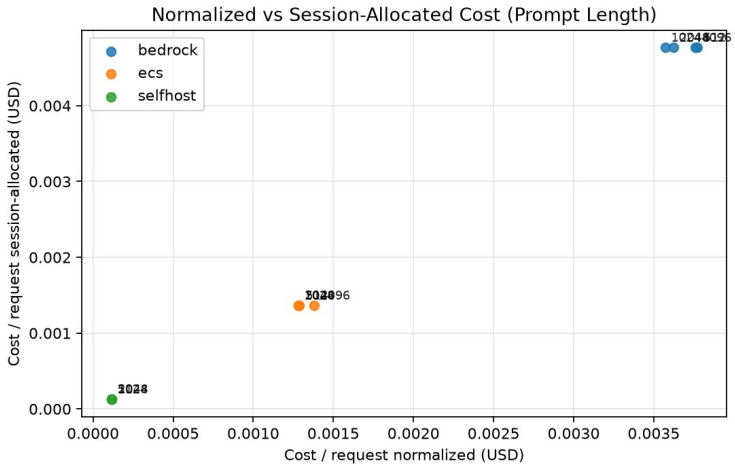

Fig. A4. Normalized vs. session-allocated cost per request, prompt-length sweep.

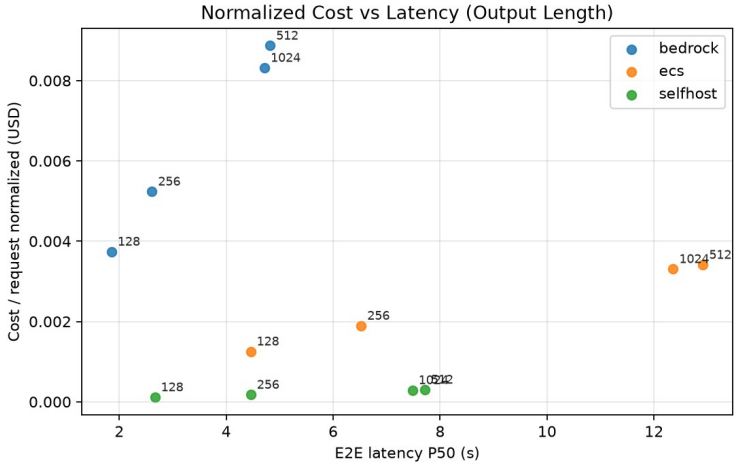

Fig. A5. Normalized cost per request vs. median end-to-end latency, output-length sweep.

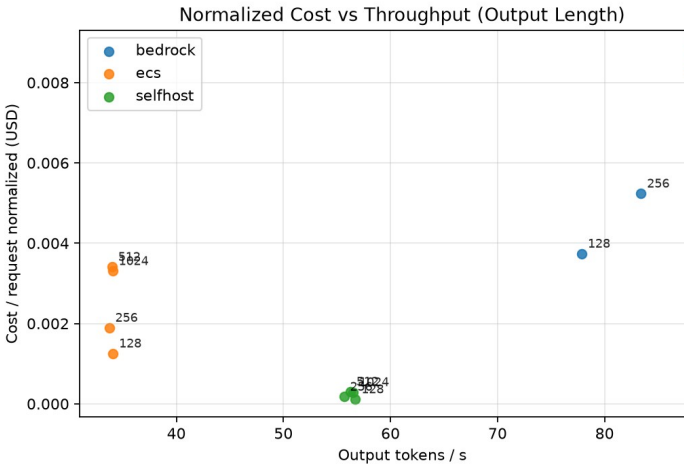

Fig. A6. Normalized cost per request vs. output-token throughput, output-length sweep.

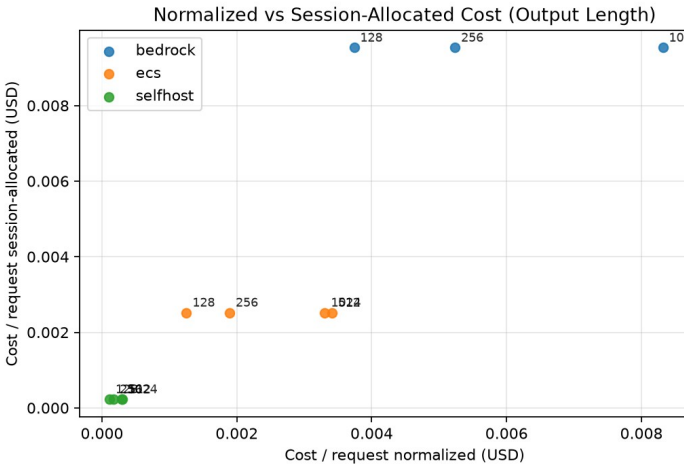

Fig. A7. Normalized vs. session-allocated cost per request, output-length sweep.